MDPI

Article

Comparison of Convolutional and Recurrent Neural Networks for Mapless Navigation in Domestic Service Robots

Abstract

Domestic service robot navigation traditionally relies on geometric maps, localization techniques such as Kalman or Particle Filters, path planning algorithms such as A* or RRT, and obstacle avoidance methods. While these approaches have demonstrated high effectiveness, they often struggle in dynamic and unstructured environments where the layout changes due to moved furniture or newly introduced objects, requiring re-mapping or system recalibration. In contrast, mapless navigation avoids the need for an explicit representation of the environment. Recent advances in Deep Neural Networks (DNNs) and Reinforcement Learning (RL) have made mapless navigation a promising alternative for robust robot navigation. In this paper, we propose two DNN-based navigation architectures, a convolutional neural network (CNN) and a recurrent neural network (RNN), that generate navigation commands directly from observations encoded in a Local Obstacle Matrix (LOM). The proposed models are compared with three traditional approaches: the Pumas Navigation System (PNS), which combines A* path planning, Monte Carlo Localization, and Potential Fields; a Finite State Machine (FSM) controller; and Artificial Potential Fields (PF). Experiments were conducted in both known and unknown environments. The results show that the PNS outperforms deep neural networks in known environments. However, it fails to generalize to unknown environments, even when they have similar shapes and layouts. In contrast, the proposed architectures achieve superior performance in unknown environments without retraining or parameter adjustment, and generalize effectively across different robot platforms.

Keywords: visual mapless navigation; mobile robots; deep learning; convolutional neural networks; recurrent neural networks

1. Introduction

Navigation is a fundamental capability for domestic service robots (DSRs). Classic navigation techniques involve three main tasks [1–5]: mapping, localization, and path planning. This approach requires a structured representation of the environment—known as a geometric map—which uses a coordinate system to encode the positions of obstacles and free space with specific shapes and dimensions. Path planning using a geometric map is highly effective as long as all environmental characteristics can be captured by the robot's sensors. Nevertheless, in crowded and dynamic environments, map-based navigation often encounters significant difficulties [6,7].

Navigation can be categorized into two main types: global and local navigation. Global navigation, which involves path planning, is typically performed using algorithms such as A*, Artificial Potential Fields (APF), Rapidly-exploring Random Trees (RRT), and Dijkstra's algorithm, among others [8,9]. However, building and maintaining maps can be time-consuming, computationally expensive [7,10], and impractical in dynamic scenarios.

Received:
Revised:
Accepted:
Published:

Copyright: © 2026 by the authors. Submitted to *Robotics* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

On the other hand, local navigation focuses on obstacle avoidance or reactive behaviors, they have lower computational complexity and offer faster response times. However, these techniques are difficult to generalize, and purely local approaches often struggle to navigate complex environments where they can become trapped in local minima.

Autonomous mobile robots must operate effectively in complex, unstructured, and previously unseen environments. Mapless navigation provides a robot with the capability to navigate unknown environments toward a specified goal without requiring an explicit geometric map [11]. Recent studies have focused on deep reinforcement learning (DRL) techniques to learn control policies for mapless navigation, demonstrating significant improvements in robotic motion planning. However, DRL still faces challenges, such as the tendency of robots to become trapped in local minima.

In this work, we present a comparison of CNN and RNN architectures for learning navigation behaviors against two conventional approaches—APF and FSM—in both known and unknown environments. Section 2 presents related work on mapless navigation. Section 3 presents the proposed mapless navigation framework, including the Local Obstacle Matrix (LOM) and the deep neural network architectures (CNN and RNN) that generate velocity commands from local perception and target information. Subsections 3.2 and 3.3 describe the proposed architectures in detail.

A critical challenge in mapless navigation is characterizing how differences between environments relate to the generalization of a trained model. To investigate this issue, we introduce an environment similarity metric based on the Wasserstein distance to quantify structural differences between environments. Section 5 formalizes the metric and evaluates its numerical stability and sensitivity to physical scale. Its relationship with navigation performance must be assessed using experimental outcomes.

By mapless, we mean that the robot does not maintain or require a global geometric map (occupancy grid, costmap, or metric representation of the environment) for path planning. The robot relies solely on local perception (the LOM, which is a local occupancy representation of the immediate surroundings) and a target vector specifying the relative position of the goal. This is in contrast to map-based systems that require a pre-built global map and localization.

We acknowledge that classical local planners such as Dynamic Window Approach (DWA) and Potential Fields (PF) are computationally efficient and well-established. However, they require manual parameter tuning that is often environment-specific and can suffer from local minima in complex layouts. Learning-based approaches, while requiring an upfront training cost, offer a compelling alternative: once trained, they can be deployed in previously unseen environments without retuning, potentially offering better generalization. Rather than aiming to universally replace classical methods, this paper proposes a navigation system designed to generalize effectively to unknown environments without relying on a geometric map, while systematically evaluating whether these generalization benefits justify the training overhead for domestic service robots.

2. Related Work

The robot's ability to navigate also requires having flexibility and adaptability in unknown and dynamic environments, where classic navigation approaches face difficulties. Many studies have addressed mapless navigation from different perspectives [12], including insect-inspired visual orientation methods [13], landmark-based, semantic navigation, deep learning models [14], RL and DRL approaches[15–17]. In more recent works, mapless navigation has made advances to increase robustness, adaptability, and deployment efficiency, especially in real-world indoor or semi-structured scenarios[18].

Recent advances addressing various aspects of mapless navigation can be categorized as follows. Vision-based navigation approaches use cameras or multimodal sensory inputs to guide robot motion. For instance, Machkour et al. [19] proposed a monocular-based navigation system leveraging multiple deep learning models to achieve reliable navigation with limited sensor inputs, using a PID controller to adjust robot motion but requiring dynamic obstacle handling. Lu et al. [20] present a novel insect-inspired embodied navigation model (differential-MB) for visual route following, which uses the difference in familiarity between left and right visual fields to steer a robot. However, their approach still requires hand-tuning and improvement in highly dynamic environments. These biologically inspired methods tend to excel in environments where computational resources or sensory data are limited; however, they may struggle with scalability or complex semantic reasoning. Li et al. [21] show experiments in the AI2-THOR simulation that combine scene priors/spatial relationships and meta-learning for improving generalization in visual navigation tasks. The policy's dependence on semantic object classes means unknown object categories might be problematic.

Learning-based control methods, particularly those using reinforcement learning (RL), have shown promise for enabling autonomous navigation without explicit maps [22]. Zhang et al. [23] proposed an improved Deep Deterministic Policy Gradient algorithm for mapless path planning, demonstrating enhanced adaptability to dynamic and unknown settings. Other RL-based frameworks, such as hierarchical RL systems for congestion estimation [24], have emphasized safety and scalability, tackling real-time navigation challenges in complex scenarios. Despite their success, RL-based methods often require substantial training data and may suffer from poor generalization when transferred to unseen environments.

Finally, vision-language and cognitive systems are emerging as promising directions to bridge low-level perception and high-level semantic understanding [25]. These models integrate natural language understanding with visual inputs to facilitate more flexible and context-aware navigation. However, integrating such models into real-time robotic systems remains an open challenge, with ongoing research addressing issues of latency, interpretability, and robustness.

Each approach offers distinct advantages, including robust generalization across diverse environments, computational efficiency suitable for real-time deployment, and the ability to handle complex perceptual and semantic reasoning tasks. Nonetheless, challenges remain in simultaneously achieving all three objectives, as methods optimized for speed may sacrifice generalization, while those designed for broad adaptability often incur higher computational overhead. Hybrid approaches [26,27] combine the strengths of different methods to achieve reliable and scalable mapless navigation in diverse real-world settings, offering a promising direction for future research.

3. Navigation System

We aim to build a navigation system where the robot requires only information about free and occupied space in its immediate surroundings. The Local Obstacle Matrix (LOM) is a small occupancy grid obtained from the LiDAR sensor and a 2D projection of a point cloud, which are used to generate a representation of the perceived obstacles (see Subsection 3.1); the LOM covers a square area in front of the robot. The target vector (d, θ) represents the relative position of the goal with respect to the robot's frame. These variables are the distance and angle in polar coordinates of the goal point. In Figure 1, the target vector and the LOM generation process are shown. Both serve as inputs to the proposed neural network architectures, and the outputs consist of velocity commands in the form (v_x, v_y, ω), where v_x and v_y are the linear velocities along the robot's x and y axes,

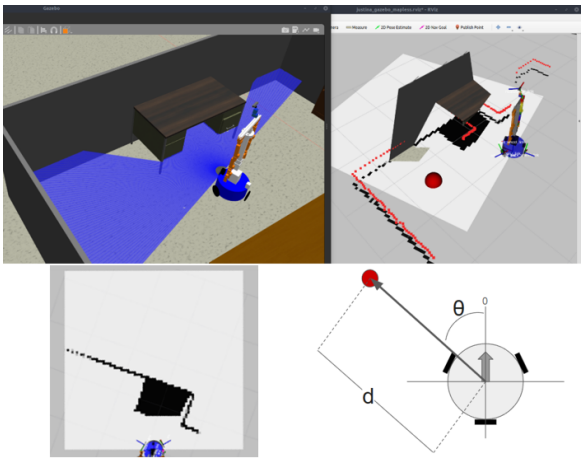


Figure 1. Local environment signals: Local Obstacle Perception Matrix (bottom left) and relative position vector of the target (bottom right).

respectively, and ω is the angular velocity around the robot's vertical axis. In Figure 2 a general scheme of the navigation behavior system is presented.

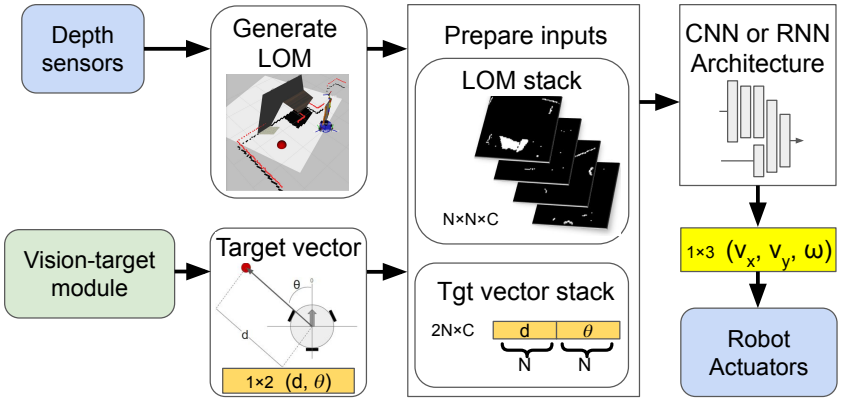


Figure 2. General navigation behavior system scheme.

3.1. The Binary Local Obstacle Matrix (LOM)

The binary LOM can be generated using various depth sensors or estimators, such as RGB-D cameras, ultrasonic sensors, LiDAR, 3D point clouds, or neural network–based depth estimators. In this work, we use two complementary sensors: an RGB-D camera mounted on the robot's head provides point cloud data, while a 2D LiDAR sensor located on the robot's base provides laser scans. We fuse both sources to build a richer representation of the surroundings. The LOM is an $N \times N$ matrix at ground level, where each cell encodes the presence of an obstacle. For each cell, we project the corresponding 3D points from the point cloud and also integrate the LiDAR readings that fall into that cell. The final value of a cell is set to 100 if either sensor detects an obstacle, and 0 otherwise.

The LOM is treated as a single-channel binary image. Its two values represent free and occupied space; changing the occupied-cell value modifies the numerical input scale without changing the obstacle geometry. To select the occupied-cell value, we compared the 0/1, 0/100, and 0/255 encodings using RNN models trained with different random seeds and the same dataset and training hyperparameters. Among the selected models, 0/100 achieved the lowest average validation mean squared error (MSE), 0.075609, compared with 0.084249 for 0/1 and 0.085079 for 0/255.

To assess the use of obstacle information, we permuted complete LOM sequences between validation samples while keeping the target vectors, reference velocity commands,

and model weights fixed. The MSE increased by 74.29% with 0/100, compared with 4.80% with 0/1 and 59.96% with 0/255. This sensitivity indicates that the 0/100 models use obstacle information together with the target vector to predict velocity commands.

In navigation tests for this encoding comparison, with obstacles concentrated on the left or right, the goal was reached without collisions in 70% of the cases with 0/100, compared with 10% with 0/1 and 50% with 0/255. These results support the selection of 0/100 for the evaluated configuration.

3.2. Proposed CNN Architecture

Convolutional neural networks (CNNs) [28] are widely used in computer vision tasks. The early layers of a neural network are typically convolutional and are used to extract features from the input data. Adding several layers increases the network's level of abstraction, and the number of layers depends on the complexity of the dataset. CNNs exhibit a certain degree of invariance to translation, scaling, and input distortions, which contributes to their robustness. Each convolutional layer has a set of filters, or channels, that determine the depth of the generated feature map. Convolutions effectively preserve spatial relationships; however, to provide some ability to learn spatio-temporal features, we propose using a sequence of binary images stacked as channels in the main input of our architectures.

Our neural network has two inputs: the first is our binary LOM, and the second is an auxiliary input (the target vector). The auxiliary input is provided at a later stage because we first want to extract features from the LOM independently. The first input can be interpreted as an $N \times N$ image. To incorporate temporal information into the learning process, C consecutive frames are provided to the network, stacked as channels forming a single input tensor of dimensions$N \times N \times C$. Each of these C frames has an associated target vector(d, θ), which together form the auxiliary input sequence. Samples are stored sequentially to preserve the frame order. During data selection in the training phase, packets of C channels are obtained before shuffling the samples. Thus, each sample is not a single image but rather a sequence of C frames, this approach accelerates training convergence.

The proposed architecture consists of three convolutional layers with ReLU activation; to reduce spatial dimensionality, max pooling is applied to the second and third layers. On the second layer, batch normalization is applied to stabilize learning. The resulting feature map is flattened, producing a high-dimensional feature vector that encodes the spatial and temporal information from the LOM sequence. This vector is then concatenated with the auxiliary input (the target vector) in a late-fusion fashion [29] (specifically, feature-level fusion), where the two information streams are processed independently before being combined at a deeper stage of the network. Because the flattened feature map from the convolutional layers has dimensionality proportional to N^2, whereas the target vector (d, θ) has only two components, direct concatenation would disproportionately weight the LOM-derived features. To prevent the auxiliary input from being marginalized during training and ensure its contribution remains relevant for the final regression task, the vector (d, θ) is replicated N times, as expressed in Equation (1). The resulting concatenated vector is then passed through two fully connected (FC) layers (each with dropout). The output layer consists of three tanh neurons that provide the learned three control robot velocities (v_x, v_y, ω). Full architecture is shown in Figure 3.

$$\text{rep_tgt} = \left[d_{i,c}, \theta_{i,c} \right]_{i=1,\ldots,N;\, c=1,\ldots,C} \in \mathbb{R}^{2NC} \tag{1}$$

For the specific implementation, we use $N = 80$ and $C = 4$, resulting in an input tensor of size $80 \times 80 \times 4$. Three convolutional layers with 3×3 kernels (16, 32, and 8

filters) and ReLU activation, followed by max pooling (5×5, stride 3), produce a flattened feature vector of 288 dimensions. The target vector replicated 80 times (160 dimensions) is concatenated, forming a 448 dimensional input to two fully connected layers (120 and 80 neurons) with dropout rates of 0.2 and 0.15, respectively. The output layer uses tanh activation to produce velocity commands (v_x, v_y, ω), normalized to $[-1, 1]$.

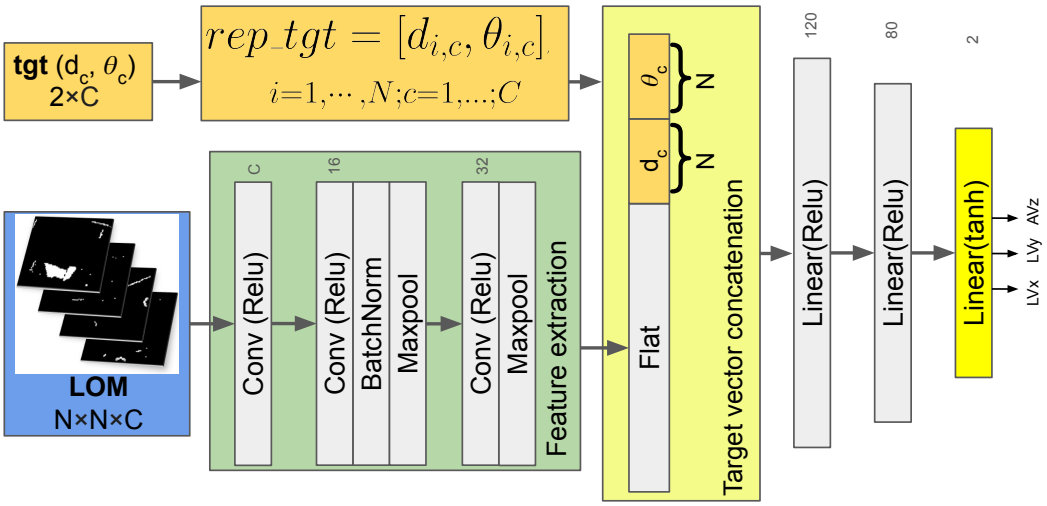


Figure 3. CNN architecture with C sequential frames and targets; target vector is replicated N times and concatenated with extracted features.

3.3. Proposed RNN Architecture

Recurrent Neural Networks (RNNs) gained prominence in the 1990s due to their ability to retain information from previous inputs through feedback connections [30]. Their feedback connections allow an inherent capacity for temporal memory, making them particularly well-suited for tasks in which temporal order and context are critical, such as language processing, music analysis, and robotics [31]. Applications of RNNs include speech recognition, machine translation, musical sequence generation, and autonomous decision-making in robots and self-driving vehicles.

To maintain compatibility within our system, our recurrent neural network architecture shares the same input and output structure as the previous CNN architecture. The first layer is a convolutional layer with max pooling to reduce feature dimensionality. The resulting feature channels are flattened separately and concatenated with the target vector (d, θ) in a different way than in the CNN. Instead of obtaining a single vector by flattening all features and concatenating it with a repeated target vector, we obtain C flattened feature vectors, each concatenated with its respective repeated target vector. After concatenation, these C vectors serve as input to our recurrent block, which consists of four simple recurrent layers (with tanh activation). Each RNN cell (see Figure 4) reuses hidden state information, and the cells are interconnected in a many-to-one configuration. The output from the recurrent block is a feature vector that passes through the three fully connected (FC) layers, which generate the control signals. The full architecture is shown in Figure 5.

Unlike the CNN, the RNN model operates with $N = 80$ and $C = 2$, producing an input tensor of size $80 \times 80 \times 2$. A single convolutional layer with 8 filters (3×3 kernels) and ReLU activation, followed by max pooling (7×7, stride 3), produces a flattened feature vector of 4608 dimensions per channel. The target vector replicated 80 times (160 dimensions) is concatenated per channel, forming a 4768 dimensional input vector for each of the C channels. These vectors are processed by a recurrent block with four RNN layers, each with 350 hidden units and tanh activation, operating in a many-to-one configuration.

The recurrent block outputs a 350 dimensional feature vector from the final time step, which is then passed through three fully connected layers (120, 80, and 3 neurons) with dropout (0.2 and 0.15) and tanh activation to produce the velocity commands (v_x, v_y, ω), normalized to $[-1, 1]$.

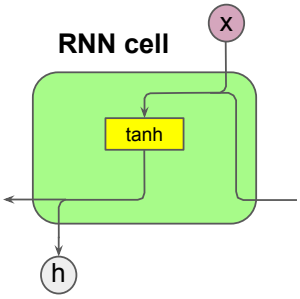


Figure 4. An RNN cell implemented in our recurrent block.

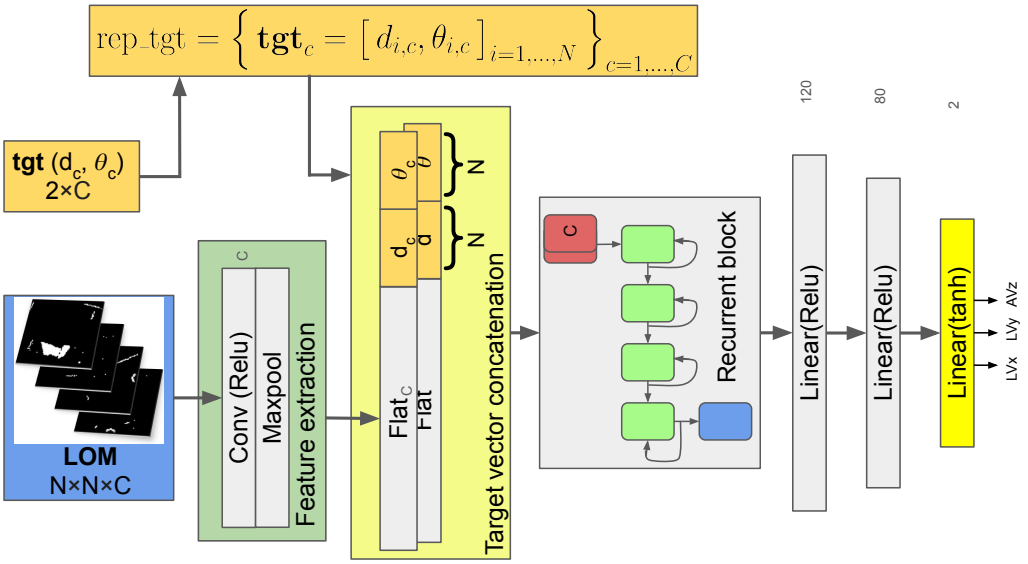


Figure 5. RNN architecture. The recurrent block receives C input vectors, each obtained by concatenating a feature vector with its corresponding repeated target vector (d, θ).

4. Training of Neural Network Architectures

Our proposed neural network models—the CNN and RNN (see Subsections 3.2 and 3.3)—employ supervised learning and therefore require labeled data. Because the network outputs are continuous velocity commands, the learning task is formulated as a regression problem. We train the models by minimizing the mean squared error (MSE) between the predicted velocities and the ground-truth commands recorded during data collection. The hyperparameters used for training both models are summarized in Table 1.

For the experiments, six different environments were generated using a custom generator developed by us (see Subsection 4.2). Three of these environments were used for data collection and are therefore considered "known" environments, while the remaining three are treated as "unknown" environments. Data collection was carried out using a script that validates and stores the labels while the robot navigates through the environment toward randomly selected target objectives inside the navigable area, as described in Subsection 4.3. Navigation can be executed either manually by a human operator (without requiring a map) or autonomously through a navigation system. In our case, to avoid introducing bias, we employed our PNS (see Subsection 4.1).

Parameter	CNN Value	RNN Value
C (stacked frames)	4	2
Batch size	8	3
Learning rate	1×10^{-4}	6×10^{-4}
Epochs	20	20
Patience	5	5
Optimizer	Adam	Adam
Loss function	MSE	MSE
Convolutional layers	3	1
Filters	16, 32, 8	8
RNN hidden units	—	350
RNN layers	—	4

Table 1. Hyperparameters for CNN and RNN training.

4.1. The Pumas Navigation System (PNS)

The PNS [32] is the standard map-based navigation system in our robots. PNS implements the A* algorithm with a cost function that evaluates collision risk during path planning; calculated paths have a configurable tradeoff between safety and distance. A particle filter with ROS implementation is used for localization and mapping. A finite state machine coordinates obstacle avoidance behavior using potential field and general planning.

4.2. Random Map Generator

We developed a Blender-based random map generator to create 3D environments with configurable size and complexity. The generator uses a procedural subdivision algorithm that starts with a rectangular plane and iteratively applies loop cuts to create interconnected rooms. The number of rooms is determined by the area of the plane and a complexity parameter. The meshes generated are imported into Gazebo, where a script randomly places furniture assets such as chairs, tables, desks, and shelves.

For the six environments used in our experiments (E1–E6), we configured the generator with the following parameters. The overall map dimensions were constrained to ranges of 5–10 meters in both the x and y axes. A complexity parameter of 3 was selected to generate environments with moderate complexity and 2 rooms per map, depending on the random seed. The number of obstacles per room was also controlled by the complexity parameter, producing between 10 and 25 furniture items per room.

We used the same generator to produce 30 environments (E1–E30) with different layouts, physical sizes, and complexity levels (C2–C30) for the analysis of environmental similarity. The navigation experiments reported in this paper were conducted in six environments (E1–E6), reflecting the scope of the original evaluation. The 30-environment analysis characterizes geometric differences across a broader set of maps; it does not, by itself, establish a relationship between the similarity metric and navigation performance.

The subdivision process introduces variability in room shapes through random cut offsets ranging from -0.8 to 0.8 as a percentage of the current room area. This ensures that the resulting corridors maintain sufficient width for the robot to navigate. The resulting geometric maps, shown in Figure 6, illustrate the occupied space in each environment; a 1 m grid is overlaid on each map to provide spatial scale. Only the E1–E3 maps were used by our PNS to collect training data, as this map-based system requires a geometric map for operation. The collected dataset was then used to train the neural networks. Importantly, the neural networks themselves **never use maps**; they operate solely on the LOM and target vector inputs, and data could also have been collected by manually operating the robot, as described in Subsection 4.3.



Figure 6. Randomly generated environments.

4.3. Dataset Collection

The curriculum learning approach [33] improves model efficiency during training by gradually increasing task difficulty. Following this principle, we initially trained the model in a single empty room to allow it to learn basic movements (turning left and right, moving forward and backward) for navigating toward a given target. Subsequently, data collection was expanded to include samples of the robot moving between rooms interconnected by a door. Unfortunately, the resulting dataset presented challenges, leading to unstable behaviors. In many cases, the trajectories followed by the trained model formed spiral paths instead of moving directly toward the target. We concluded that in empty environments, the predominantly zero values in the LOM prevented the model from effectively associating appropriate control commands with similar inputs.

To address this issue, training and testing data were automatically captured by a script using the PNS (see Figure 7 and Subsection 4.1), which enables the robot to navigate within an environment for data acquisition. The corresponding geometric maps were required for the PNS only and not for other purposes. The script loads the occupancy grid map of the current environment, extracts all free cells, and stores their coordinates in a temporary array. At each step, it randomly selects a point from this array as the new target, ensuring that the goal always lies in free space. To maintain consistency in the collected samples—so that trajectories have similar length and difficulty—targets are constrained to lie within a 3.5 m radius of the robot's current position. To eliminate the risk of collision, which would produce erroneous sensor readings and control commands, the PNS employs potential fields with lateral movements (linear velocity y). A total of 3,880 samples were collected across E1, E2, and E3 (see Figure 6).

To incorporate temporal information—particularly for the CNN model—the dataset is recorded and organized as sequences of C consecutive samples collected during navigation toward a designated target. Each sample consists of a pair comprising the LOM and target vector as input, and the corresponding command velocity sequence as output. In other words, we store each training sample as a sequence of C simple samples, where each input

pair (**LOM**, **tgt**) is associated with its corresponding output velocity vectors **v**. Formally, the label for sample i is defined as

$$\text{Label}^{(i)} = \left((\mathbf{LOM}^{(i)}, \mathbf{tgt}^{(i)}), \mathbf{v}^{(i)}\right) \tag{2}$$

where:

$\text{LOM}^{(i)} \in \{0, 100\}^{N \times N \times C}$: sequence of C binary LOM frames for sample i.
$\mathbf{tgt}^{(i,c)} = (d^{(i,c)}, \theta^{(i,c)}) \in \mathbb{R}^2$: sequence of C target vectors for sample i.
$\mathbf{v}^{(i,c)} = (v_x^{(i,c)}, v_y^{(i,c)}, \omega^{(i,c)}) \in \mathbb{R}^3$: sequence of C command vectors for sample i, where v_x and v_y are the linear velocities along the robot's x and y axes, respectively, and ω is the angular velocity around the robot's vertical axis.
N: LOM pixel size.

We tested multiple hyperparameter combinations and selected those that stabilized training and produced consistent results. For our CNN, we selected a batch size of 8, meaning that each training batch consists of 8 samples, with each sample corresponding to a 4-channel ($C = 4$) LOM array of consecutive frames. For the RNN model, we use a batch size of 3, with each sample corresponding to a 2-channel ($C = 2$) LOM array. The LOM is a binary N×N pixel matrix encoding a squared area in front of the robot. A 4 m^2 area works well in different scenarios. Using a larger area may reduce training stability, while a smaller area reduces the capability to detect obstacles and thus increases the collision risk. The size of N in pixels is 80 and is defined by the resolution of the local occupancy projection calculated as follows: $N = m \times Res$.

The underlying control loop of the PNS, which includes the potential fields for obstacle avoidance, operates at 30 Hz. While the robot navigates to each new target, the input signals (LOM and target vector, described in Subsection 3.1) and their corresponding labels $\mathbf{v} = (v_x, v_y, \omega)$ are recorded at 5 Hz. This sampling rate ensures that consecutive samples are sufficiently different, preventing redundant samples that would add little information to the learning process. It also captures the net effect of the potential fields rather than momentary fluctuations, ensuring the dataset reflects meaningful navigation behavior instead of high-frequency noise. The data are stored in a Python dictionary as an .npz file while at least one component of data_Y exceeds 0.1; otherwise, the sample is discarded (i.e., when the robot remains stationary).

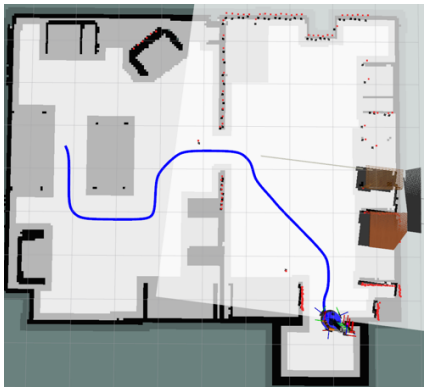


Figure 7. Automated data collection process using the PNS.

5. Environment Similarity Metric

The core hypothesis of this work is that the geometric structure of an environment dictates the difficulty of zero-shot generalization for a mapless navigation policy. To quantify this structural difference, we model each environment as a discrete probability distribution of obstacles and compute the optimal transport cost between them.

5.1. Definition of the Metric

Environment Representation.

Let an environment $\mathcal{E}$ be represented by a binary occupancy grid $\mathcal{M}$, where cells are either free or occupied. We extract the set of metric coordinates of all occupied cells, denoted as $\mathcal{P} = \{p_i = (x_i, y_i)\}_{i=1}^{|\mathcal{P}|}$, where x_i and y_i are expressed in meters relative to a common global origin. To ensure the metric is invariant to the absolute placement of the map within the grid, we align all maps by their geometric center prior to extraction.

Since a full occupancy grid contains tens of thousands of points, we treat the environment as an empirical probability measure μ over its occupied cells:

$$\mu = \frac{1}{|\mathcal{P}|} \sum_{i=1}^{|\mathcal{P}|} \delta_{p_i} \tag{3}$$

where δ_{p_i} is the Dirac delta function at position p_i. To make the computation tractable for high-resolution grids, we subsample a fixed number N of points uniformly at random from each map, where N is referred to as the *sampling density*. Throughout this work we use $N = 4000$ points per environment, yielding a sparse empirical distribution $\hat{\mu}$ over the subsampled cells.

Wasserstein Distance and Sinkhorn Regularization.

Given two environments $\mathcal{A}$ and $\mathcal{B}$ with empirical distributions $\hat{\mu}_A$ and $\hat{\mu}_B$, we quantify their structural dissimilarity using an entropy-regularized optimal transport cost related to the 1-Wasserstein distance (Earth Mover's Distance). This cost quantifies the effort required to transport the obstacle distribution of $\mathcal{A}$ into that of $\mathcal{B}$.

To compute this efficiently while maintaining differentiability and numerical stability, we adopt the entropy-regularized optimal transport formulation (Sinkhorn distance). Let $C \in \mathbb{R}^{N \times N}$ be the cost matrix, where $C_{ij} = \|p_i - p_j\|_2$ is the Euclidean distance between points $p_i \in \mathcal{A}$ and $p_j \in \mathcal{B}$. The Sinkhorn distance is defined as:

$$d_{\text{Sinkhorn}}(\mathcal{A}, \mathcal{B}) = \min_{\gamma \in \Pi(\hat{\mu}_A, \hat{\mu}_B)} \sum_{i,j} \gamma_{ij} C_{ij} - \lambda H(\gamma) \tag{4}$$

where $\Pi(\hat{\mu}_A, \hat{\mu}_B)$ is the set of all transportation plans (joint probabilities with marginals $\hat{\mu}_A$ and $\hat{\mu}_B$), $H(\gamma) = -\sum_{i,j} \gamma_{ij} \log \gamma_{ij}$ is the entropy of the transport plan, and $\lambda = 0.001$ is the entropic regularization parameter. A smaller λ yields a result closer to the exact EMD, but requires more iterations to converge. We use $\lambda = 0.001$ with a maximum of 200 iterations.

Global Scale Normalization.

To ensure that the metric values are interpretable and independent of the absolute scale of the dataset, the cost matrix is normalized by the maximum pairwise distance across all sampled points in the dataset, denoted as s_{global}. This normalization bounds the Wasserstein distances to the range $[0, 1]$, allowing direct comparison across different datasets and preventing numerical overflow during the Sinkhorn iterations.

Bootstrap Averaging.

Because the empirical distributions depend on the random subsampling of N points, a single distance computation is subject to sampling noise. To obtain a robust estimate, we perform $B = 5$ independent bootstrap resamples. The final reported distance $\hat{d}(\mathcal{A}, \mathcal{B})$ is the mean over the B runs, and the standard deviation across these runs summarizes resampling variability; it is not, by itself, a confidence interval.

5.2. Validation of the Metric

To characterize the proposed environment similarity metric, we evaluated all 30 procedurally generated environments, yielding 435 unique pairwise comparisons. The

analysis examines numerical stability under point subsampling and sensitivity to physical scale; it does not yet validate the metric as a predictor of navigation performance.

Statistical Robustness and Dynamic Range.

As detailed in Table 2, the pairwise distances range from 0.0337 (nearly identical layouts) to 0.1733 (highly distinct topologies), providing a dynamic range of 5.14×. The ranked pairs in Tables 5 and 4 illustrate the range of structural differences measured across the 30 environments. Furthermore, the metric is highly stable: the mean coefficient of variation (CV) across all pairs is 1.20% over 5 bootstrap resamples with a sampling density of $N = 4000$ points per environment. To evaluate the sensitivity to point cloud density, we performed a cross-validation using $N = 2000$ points. The rank correlation between both configurations exceeded 0.99, and the dynamic range remained virtually unchanged (5.05×). While $N = 2000$ is computationally sufficient, we report the $N = 4000$ configuration for maximum statistical fidelity.

Scale Sensitivity vs. Topological Similarity.

A critical challenge for similarity metrics is distinguishing between visual topological similarity and physical metric scale. To address this, we designed a controlled reference case. Environment E07 was generated with the exact same furniture layout, angles, and relative positions as E05, but scaled to a larger physical dimension.

As shown in Table 3, despite their visual similarity, the metric yields a distance of 0.1185 ± 0.0007 between E05 and E07, placing them in the top 15% of most dissimilar pairs. In contrast, environments with different topologies but identical physical scales (e.g., E05 and E03) yield a distance of only 0.0472. This indicates that the metric is sensitive to physical distances between obstacles (in meters), in addition to layout differences. Whether these distances predict navigation generalization requires a separate analysis against navigation outcomes. A policy trained in a smaller room may fail in a visually identical but larger room due to different spatial constraints and required trajectory lengths.

Table 2. Statistical summary of the 435 pairwise Wasserstein distances across 30 environments. N denotes the sampling density (points per environment).

Metric	**Configuration** $N = 4000$	**Configuration** $N = 2000$
Distance range	$0.0337 - 0.1733$	$0.0349 - 0.1762$
Dynamic range (max/min)	$5.14\times$	$5.05\times$
Mean Coefficient of Variation (CV)	1.20%	1.75%
Spearman rank correlation	> 0.99	

Table 3. Controlled scale validation. E05 and E07 share identical visual layouts but differ in physical scale. Distances are reported as mean ± standard deviation. N denotes the sampling density.

Pair	**Condition**	**Distance** ($N = 4000$)	**Distance** ($N = 2000$)
E05 vs. E03	Same scale, diff. layout	0.0472 ± 0.0006	0.0481 ± 0.0002
E05 vs. E07	**Diff. scale, same layout**	$\mathbf{0.1185 \pm 0.0007}$	$\mathbf{0.1192 \pm 0.0010}$
E05 vs. E10	Diff. scale, diff. layout	0.1733 ± 0.0006	0.1762 ± 0.0017

Ranked environment pairs. Tables 4 and 5 report the ten most structurally distinct and the ten most structurally similar pairs, respectively, according to the mean normalized distance computed with $N = 4000$ points per environment. The reported σ is the standard deviation across the five resampling runs.

6. Experimental Setup

Once our neural network models were trained, the robot was placed in environments E1 through E6. Regardless of its initial position (no localization is required), a target point

Table 4. Top-10 most structurally distinct environment pairs.

Rank	Pair	Complexity	Distance ($\hat{d} \pm \sigma$)
1	E05 vs E10	C2 vs C3	0.1733 ± 0.0006
2	E10 vs E28	C3 vs C30	0.1688 ± 0.0016
3	E05 vs E09	C2 vs C3	0.1655 ± 0.0012
4	E01 vs E28	C2 vs C30	0.1634 ± 0.0021
5	E09 vs E28	C3 vs C30	0.1612 ± 0.0018
6	E04 vs E28	C2 vs C30	0.1602 ± 0.0015
7	E07 vs E28	C3 vs C30	0.1586 ± 0.0028
8	E05 vs E23	C2 vs C8	0.1488 ± 0.0006
9	E05 vs E19	C2 vs C5	0.1451 ± 0.0012
10	E01 vs E05	C2 vs C2	0.1444 ± 0.0012

Table 5. Top-10 most structurally similar environment pairs.

Rank	Pair	Complexity	Distance ($\hat{d} \pm \sigma$)
1	E19 vs E25	C5 vs C10	0.0337 ± 0.0002
2	E01 vs E04	C2 vs C2	0.0350 ± 0.0003
3	E21 vs E29	C8 vs C30	0.0416 ± 0.0011
4	E18 vs E20	C5 vs C5	0.0419 ± 0.0012
5	E16 vs E29	C5 vs C30	0.0444 ± 0.0008
6	E22 vs E27	C8 vs C10	0.0461 ± 0.0004
7	E03 vs E05	C2 vs C2	0.0472 ± 0.0006
8	E09 vs E19	C3 vs C5	0.0480 ± 0.0008
9	E15 vs E26	C5 vs C10	0.0484 ± 0.0005
10	E02 vs E14	C2 vs C3	0.0498 ± 0.0009

was randomly selected for navigation. The trained model waits for C consecutive frames as input to perform inference, stacking the LOM and target vector as depicted in Figure 2. The output of the trained model consists of the velocity commands (v_x, v_y, ω) that our robot must command to its actuators. Often, the robot demonstrated the ability to evade dynamic obstacles.

Table 6 summarizes the use of environments for training (tuning, configuring) and testing across maps E1 to E6.

6.1. Test of the Pumas Navigation System (PNS) in Known and Unknown Environments

Traditional map-based navigation systems have been shown to outperform many mapless navigation techniques in previously known environments, where an updated map is provided. However, they lack the ability to generalize navigation when the environment suffers significant changes. This limitation is proven in the following experiment.

We start with environment E1 to navigate with our map-based PNS, using the corresponding map E1. As expected, the navigation performance was very high. In a second stage, we deployed our robot in environment E4, but using the map from E1 instead. Considering some similarity in space between E1 and E4, and even though the PNS employs potential fields to obstacle avoidance, navigation performance drops drastically, and was unable to successfully navigate to a new target provided. The experiment highlights the need to remap every new unseen environment. The results are summarized in Table 7.

6.2. Mapless Navigation Techniques Comparison

All environments generated (see Subsection 4.2) were used to test the navigation performance of each methodology, except for the map-based navigation system, which was tested only in E4 environment. For the neural network models, E1, E2, and E3 were used to

Used for / Map	E1	E2	E3	E4	E5	E6
Training CNN	x	x	x			
Training RNN	x	x	x			
PF Constant Tuning		x				
FSM Parameter Selection				x		
PNS Parameter Selection	x					
CNN Testing	x	x	x	x	x	x
RNN Testing	x	x	x	x	x	x
PF Testing	x	x	x	x	x	x
FSM Testing	x	x	x	x	x	x
PNS Testing				x		

Table 6. Usage of environments for training (or configuring/tuning) and testing. An x indicates that the environment was used for that purpose. For the neural network models (CNN and RNN), environments E1–E3 are *known* (used for training), while E4–E6 are *unknown* (not seen during training).

Navigation system	Env E1 with map E1	Env E4 with map E1
Reach the Target	98%	26%
Collisions	0	26

Table 7. PNS tested in known (E1) and unknown (E4) environments.

collect the dataset; thus, they are considered previously known environments, while E4, E5, and E6 are considered unknown. The finite state machine (FSM) was configured in advance in environment E4 to test its performance, whereas the parameters of the potential fields (PF) were tuned in E2. Consequently, E4 and E2 are considered previously known environments for the FSM and PF baselines, respectively.

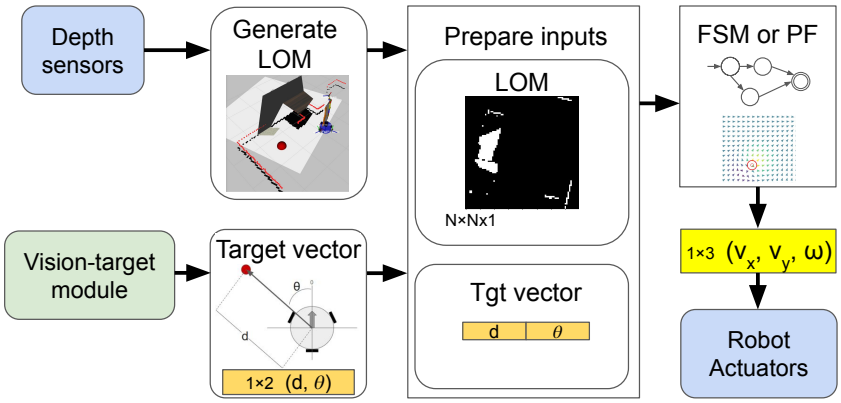


Figure 8. General navigation scheme describing the behavior of FSM and PF approaches.

The finite state machine (FSM) baseline implements reactive obstacle avoidance using laser scans and a local occupancy grid; the general navigation scheme for its behavior is shown in Figure 8. The FSM evaluates three regions—left, front, and right—as illustrated in Figure 9. Obstacle detection is performed by laser scan data, which provides distance measurements, and a local occupancy grid, which gives obstacle density in the robot's immediate vicinity. When an obstacle is detected, the FSM selects an evasion direction (left, right, or backward) based on free space availability. The front region is considered blocked if the occupancy grid shows any obstacle directly ahead. For side selection, the FSM checks both left and right using laser scans to identify free space. If both sides are blocked, the robot rotates toward the largest visible gap detected by the laser and moves backward to create space for maneuvering. If both sides are free, it selects the side with the

Algorithm 1 Pseudocode for the Finite State Machine (FSM) Baseline

```
while true do
    if no goal then
        Stop and wait for goal
    else
        Read d, θ, lidar, occupancy
        if robot is within goal tolerance then
            Stop
        else if front is free then
            Move toward the goal
        else
            s ← ChooseFreeSide(lidar, occupancy)
            if s ≠ BLOCKED then
                while front blocked do
                    Rotate toward free side s
                end while
            else
                Rotate to largest gap; move back
            end if
            Move forward a short distance
        end if
    end if
end while
```

```
function ChooseFreeSide(lidar, occupancy)
    if both sides free then return side with more space
    else if left free then return LEFT
    else if right free then return RIGHT
    elsereturn BLOCKED
    end if
end function
```

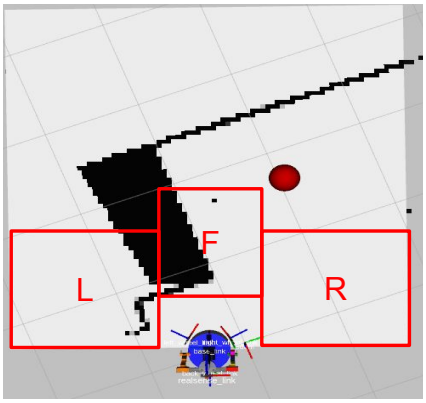


Figure 9. Perception zones used by the FSM for obstacle detection: left (L), front (F), and right (R). The red-marked areas indicate the regions evaluated to determine free space and select evasion directions.

largest continuous free space; if only one side is free, that side is chosen. When all regions are free, the robot rotates for a fixed duration toward the goal based on the initial target angle and moves forward before evaluating new sensor readings. The reactive navigation strategy implemented by the FSM is summarized in Algorithm 1.

In contrast to the rule-based FSM, the potential fields (PF) baseline follows the same general navigation scheme illustrated in Figure 8, using attractive and repulsive forces to generate velocity commands. The attractive force pulls the robot toward the goal, while repulsive forces push it away from obstacles detected by laser scans. The resultant force vector determines the robot's motion direction. The logic is detailed in Algorithm 2.

Algorithm 2 Pseudocode for the Potential Fields (PF) Baseline

while true **do**
  if no goal **then**
    Stop and wait for goal
  else
    Read target (d, θ), laser, occupancy
    if within goal tolerance **then**
      Stop
    else
      $\mathbf{F}_{attr} \leftarrow$ attractive force to goal
      $\mathbf{F}_{rej} \leftarrow$ repulsive force from obstacles
      $\mathbf{F} \leftarrow \mathbf{F}_{attr} + \mathbf{F}_{rej}$
      Compute v, w from $\mathbf{F}$
      Publish (v, w)
      if speed near zero $>$ 6 seconds **then**
        Increase repulsive gain temporarily
      end if
    end if
  end if
end while

function ATTRACTIVEFORCE(goal)
  $\mathbf{q} \leftarrow$ unit vector to goal
  return $-\eta \cdot \mathbf{q}$
end function

function REPULSIVEFORCE(laser, occupancy)
  $\mathbf{F} \leftarrow (0, 0)$
  for each laser reading with $d_i < d_0$ **do**
    $f \leftarrow \zeta \sqrt{1/d_i - 1/d_0}$
    $\mathbf{F} \leftarrow \mathbf{F} + f \cdot (\cos\theta_i, \sin\theta_i)$
  end for
  Add occupancy contribution scaled by γ
  return F
end function

At each iteration, the attractive force $\mathbf{F}_{attr}$ is computed as a unit vector pointing from the robot to the goal, scaled by $\eta = 1.5$. For each laser reading within $d_0 = 0.5$ m, a repulsive force $\mathbf{F}_{rej}$ is calculated as $f = \zeta\sqrt{1/d_i - 1/d_0}$ with $\zeta = 8.5$, and summed over all readings. The total force $\mathbf{F} = \mathbf{F}_{attr} + \mathbf{F}_{rej}$ defines the desired motion direction.

By processing the local occupancy grid, we estimate an extra repulsive component for dynamic obstacles, scaled by $\gamma = 0.4$. The linear and angular velocities are then derived from the orientation error between the robot's current heading and the desired force direction. Linear velocity follows $v = v_{max} \cdot e^{-\text{error}^2/\alpha}$ with $\alpha = 0.8$ and $v_{max} = 0.4$ m/s, while angular velocity follows $w = w_{max} \cdot \left(\frac{2}{1+e^{-\text{error}/\beta}} + 1\right)$ with $\beta = 0.2$ and $w_{max} = 0.8$ rad/s, ensuring smooth turning.

https://doi.org/10.3390/robotics1010000

An escape mechanism is included to handle situations where the robot becomes trapped in local minima, a known limitation of potential fields methods. If the robot's linear speed remains near zero for more than six seconds, it is considered stuck. The repulsive force gain is temporarily increased to 100, a value empirically determined to force the robot to reverse away from the obstacle, this high gain creates a strong evasion force that overcomes the local minimum. After a brief window of 1.5 seconds, sufficient for the robot to move away from the obstacle, the gain resets to its nominal value, allowing the robot to resume goal-directed navigation.

Both the PF implementation and the rule-based FSM serve as reactive baselines, contrasting with the learning-based CNN and RNN architectures. The parameters for both baseline methods were empirically tuned to ensure fair comparison with the proposed learning-based architectures. For the FSM, the front obstacle threshold and evasion timing were configured in environment E4, which was selected because it is small yet sufficiently complex (narrow passages) to rapidly detect and tune collision evasion behavior without requiring extensive testing cycles. For the PF method, the attractive gain η, repulsive gain ζ, and influence distance d_0 were tuned in environment E2 through systematic trials. The tuning process prioritized a balance between responsiveness to obstacles and smooth convergence to the goal, avoiding excessive oscillations or local minima entrapment.

Each test was executed by a script that generates 100 random navigation goals, each within a radius of 3 meters from the robot's current position. Goals located inside walls or on top of furniture are not valid. A time limit of 60 seconds per goal was set, meaning that if the robot does not reach the target on time, it is considered a failure. Once the robot reaches a goal within the valid time, it is considered a success. After a success or failure, a new goal is generated.

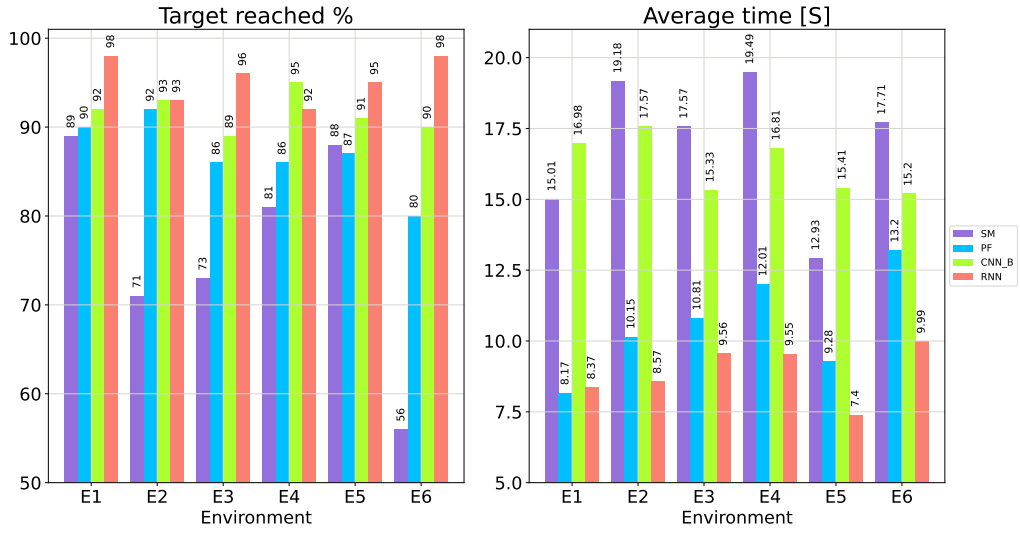


Figure 10. Successful target reach percentage (Left) and average time to reach a goal (Right). Environments with previous knowledge (training, parameters adjustment): FSM (E4), PF (E2), CNN and RNN (E1, E2, E3).

7. Results

A direct comparison with the Pumas Navigation System (PNS) highlights the limitations of map-based approaches when operating outside their training conditions. As shown in Table 7, the PNS achieves a high success rate of 98% in the known environment (E1), where an accurate map is available. This confirms the effectiveness of classical navigation pipelines when reliable prior information is provided.

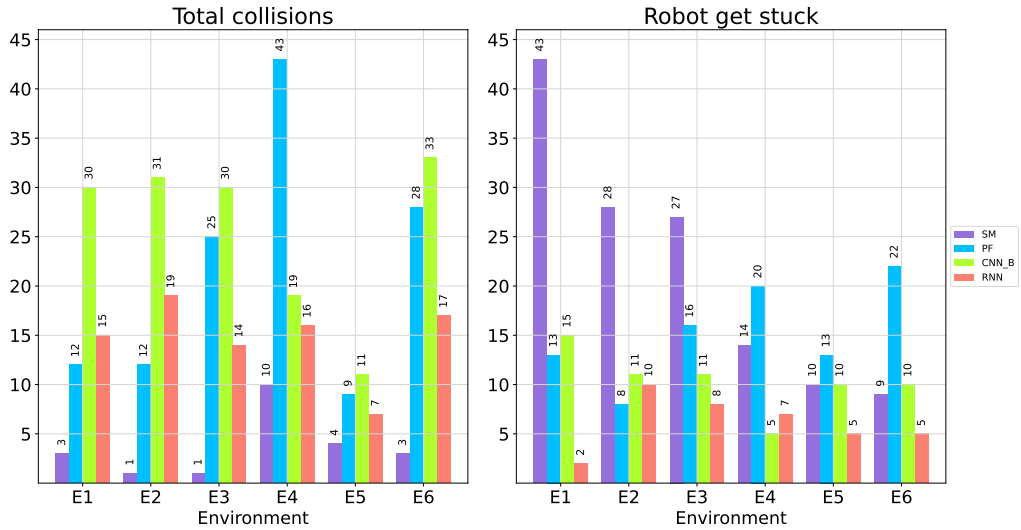


Figure 11. Collisions occurrences (Left) and failures (Right), considering not arriving on time or getting stuck by collision.

However, when deployed in an unseen environment (E4) while still relying on the map from E1, performance drops drastically to a 26% success rate, accompanied by a significant increase in collisions. Despite the use of potential fields for local obstacle avoidance, the mismatch between the assumed map and the actual environment leads to incorrect path planning decisions and unsafe behaviors.

The environment similarity metric in Section 5 provides a quantitative comparison of map geometry, but it does not by itself explain the PNS failure. In fact, E01 and E04 appear among the ten most structurally similar pairs, with a reported distance of 0.0350 in Table 5. Therefore, the low PNS success rate in E4 when using the E1 map should be interpreted as evidence of sensitivity to map mismatch in this particular experiment, not as a direct consequence of a large value of the proposed metric. A relationship between environmental distance and navigation outcomes requires additional experiments.

These results demonstrate a fundamental limitation of map-based navigation systems: their strong dependence on accurate and up-to-date environmental representations. Even when environments share similar geometric properties, small discrepancies—such as furniture placement or corridor structure—can severely degrade performance. In contrast, our mapless approaches rely solely on local perception (LOM) and target information, allowing them to adapt online to environmental variations without requiring remapping or localization.

Building on this comparison, we evaluate all navigation methods across both known and unknown environments. Our learning-based models, particularly the RNN (Subsection 3.3), consistently outperform traditional reactive methods such as finite state machines (FSM), which rely on predefined rules, and potential fields (PF), which are prone to local minima and reactive-only behaviors. These results confirm that the primary advantage of the proposed approach lies in its ability to generalize robustly to previously unseen environments.

During navigation, all collisions were counted, as they may briefly slow the robot, cause it to become stuck, or delay its arrival at the goal. These events provide insight into the robot's stability and adaptability in dynamic, cluttered environments. The success rate, defined as the proportion of goals reached within a fixed time limit, is reported in Figure 10 (Left). The corresponding average time to reach each goal is shown in Figure 10 (Right), offering a measure of efficiency for each method. Across all tested environments, collisions,

instances of the robot becoming stuck, and failures to reach targets within the time limit were observed, reflecting the challenges posed by complex layouts and narrow pathways. A summary of these events is presented in Figure 11.

To complement these quantitative measures, we generated heatmaps that provide a spatial visualization of the robot's performance in each environment. These heatmaps are derived from the recorded collision and getting-stuck events summarized in the corresponding figures. They serve as a complementary visual aid rather than a standalone quantitative metric, allowing for a qualitative identification of areas where each navigation method faced particular challenges. In the maps, red regions correspond to zones with a high frequency of collisions and getting stuck; for FSM, red areas also include locations near doors where the robot frequently exited the map boundaries. Orange regions indicate areas where the robot experienced difficulties, such as maneuvering close to obstacles or through tight turns, and yellow regions mark zones with low collision frequency but slight issues in aligning toward the target. Comparing heatmaps across methods—FSM (Figure 12), PF (Figure 13), CNN (Figure 14), and RNN (Figure 15)—clearly shows how learning-based approaches reduce collisions, avoid local minima, and maintain more consistent trajectories, even in unfamiliar environments.



Figure 12. State machine (FSM) heat map behavior. Parameter Selection E4.

8. Conclusions

We proposed two neural network architectures for generating local navigation behaviors with obstacle avoidance: a convolutional neural network (CNN) and a recurrent variant (RNN) with four recurrent layers (see Subsections 3.2 and 3.3). Unlike traditional methods—where map-based approaches may fail without an accurate or updated map—our models do not require retraining or parameter tuning, allowing them to navigate effectively even without prior knowledge of the environment. In comparison, classical reactive approaches such as Potential Fields (PF) and Finite State Machines (FSM) often rely on hand-tuned parameters and may fail in unknown settings. Although the PNS performs better in a known environment (E1), its performance drops significantly in an unseen environment (E4) when using the same map, highlighting that our main contribution is robust generalization rather than superior performance in familiar scenarios.



Figure 13. Potential fields (PF) heat map behavior. Constant Tuning E2.



Figure 14. Convolutional neural network (CNN) heat map behavior. Training E1, E2, and E3.



Figure 15. Recurrent neural network (RNN) heat map behavior. Training E1, E2, and E3.

Our experiments evaluated all methods in both previously known and unknown environments. The RNN models demonstrated strong generalization, achieving high success rates, fewer robot stuck events, and more efficient navigation than traditional approaches. Heatmaps of navigation behavior revealed specific areas where collisions or difficulties occurred, highlighting the robustness and adaptability of our learning-based models in dynamic, cluttered environments. Among the two architectures, the RNN consistently outperformed the CNN in most metrics, likely due to its ability to incorporate temporal context into decision-making.

Furthermore, we introduced and evaluated an environment similarity metric based on entropy-regularized optimal transport. The pairwise analysis across 30 environments showed a broad range of distances, stable rankings under the tested sampling densities, and sensitivity to physical scale. These results support the use of the metric to characterize geometric differences between environments. Its ability to predict navigation performance remains to be established by correlating the distances with outcomes from controlled navigation experiments.

A key challenge remains in heuristically generating a suitable dataset that maximizes performance across diverse environments. Nevertheless, the results indicate that learning-based navigation strategies offer a reliable, scalable, and flexible alternative to traditional rule-based methods, particularly in scenarios where prior knowledge of the environment is limited or unavailable.

Author Contributions:

Funding: This work was partially supported by UNAM-DGAPA under grant IN118226.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The datasets generated during the current study are available from the corresponding author on reasonable request.

Acknowledgments: Not applicable.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
LOM	Local Obstacle Matrix
PNS	Pumas Navigation System
FSM	Finite State Machine
PF	Potential Fields
DSR	Domestic Service Robot
MSE	Mean Squared Error
EMD	Earth Mover's Distance

1. Placed, J.A.; Strader, J.; Carrillo, H.; Atanasov, N.; Indelman, V.; Carlone, L.; Castellanos, J.A. A survey on active simultaneous localization and mapping: state of the art and new frontiers, 2023, [arXiv:cs.RO/2207.00254].
2. Tola, T.A.; Mi, J.; Che, Y. Mapping and localization of autonomous mobile robots in simulated indoor environments. *Frontiers* **2024**, *4*, 91–100. https://doi.org/10.11648/j.frontiers.20240403.13.
3. Valencia, R.; Andrade-Cetto, J. *Mapping, Planning and Exploration with Pose SLAM*, 1 ed.; Springer: Cham, 2018; pp. XII, 114. https://doi.org/10.1007/978-3-319-60603-3.
4. Gao, Y.; Xu, H.; Hu, M.; Liu, J.; Liu, J. Path planning under localization uncertainty. *Journal Européen des Systèmes Automatisés* **2017**, *50*, 463–484.
5. Dubé, R.; Gawel, A.; Cadena, C.; Siegwart, R.; Freda, L.; Gianni, M. 3D localization, mapping and path planning for search and rescue operations. In Proceedings of the 2016 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR), 2016, pp. 272–273. https://doi.org/10.1109/SSRR.2016.7784311.
6. Duberg, D.; Zhang, Q.; Jia, M.; Jensfelt, P. DUFOMap: Efficient dynamic awareness mapping, 2024, [arXiv:cs.RO/2403.01449].
7. Cadena, C.; Carlone, L.; Carrillo, H.; Latif, Y.; Scaramuzza, D.; Neira, J.; Reid, I.; Leonard, J.J. Past, present, and future of simultaneous localization and mapping: toward the robust-perception age. *IEEE Transactions on Robotics* **2016**, *32*, 1309–1332. https://doi.org/10.1109/TRO.2016.2624754.
8. Yan, K.; Ma, B. Mapless navigation based on 2D LIDAR in complex unknown environments. *Sensors* **2020**, *20*, 5802.
9. Gul, F.; Rahiman, W.; Alhady, S.S.N. A comprehensive study for robot navigation techniques. *Cogent Engineering* **2019**, *6*, 1632046.
10. Liu, H.; Dong, Y.; Hou, C.; Liu, Y.; Shu, Z.; Xu, S.; Lv, T. Parallel implementation for real-time visual SLAM systems based on heterogeneous computing. *Intelligence & Robotics* **2024**, *4*, 256–275.
11. Liang, J.; Wang, Z.; Cao, Y.; Chiun, J.; Zhang, M.; Sartoretti, G.A. Context-aware deep reinforcement learning for autonomous robotic navigation in unknown area. In Proceedings of the Proceedings of The 7th Conference on Robot Learning, Cambridge, MA, USA, 2023; Vol. 229, *Proceedings of Machine Learning Research*, pp. 1425–1436.
12. Guzel, M. Autonomous vehicle navigation using vision and mapless strategies: a survey. *Advances in Mechanical Engineering* **2013**, *5*, 415747.
13. Wang, H.; Wang, S.; Qian, L. An insect vision-inspired neuromorphic vision systems in low-light obstacle avoidance for intelligent vehicles. *Machine Vision and Applications* **2024**, *35*.
14. Xie, L.; Markham, A.; Trigoni, N. SnapNav: Learning mapless visual navigation with sparse directional guidance and visual reference. In Proceedings of the 2020 IEEE International Conference on Robotics and Automation (ICRA), 2020, pp. 1682–1688. https://doi.org/10.1109/ICRA40945.2020.9197523.
15. Fan, T.; Cheng, X.; Pan, J.; Manocha, D.; Yang, R. CrowdMove: Autonomous mapless navigation in crowded scenarios. *arXiv preprint* **2018**, [1807.07870].
16. Zhu, Y.; Mottaghi, R.; Kolve, E.; Lim, J.J.; Gupta, A.; Fei-Fei, L.; Farhadi, A. Target-driven Visual Navigation in Indoor Scenes using Deep Reinforcement Learning. *CoRR* **2016**.

17. Xue, H.; Hein, B.; Bakr, M.; Schildbach, G.; Abel, B.; Rueckert, E. Using deep reinforcement learning with automatic curriculum learning for mapless navigation in intralogistics. *Applied Sciences* **2022**, *12*.
18. Pushp, D.; Chen, Z.; Luo, C.; Gregory, J.M.; Liu, L. POVNav: A Pareto-optimal mapless visual navigator. In Proceedings of the Experimental Robotics; Ang Jr, M.H.; Khatib, O., Eds., Cham, 2024; pp. 250–263. https://doi.org/10.1007/978-3-031-63596-0_22.
19. Machkour, Z.; Ortiz-Arroyo, D.; Durdevic, P. Monocular based navigation system for autonomous ground robots using multiple deep learning models. *International Journal of Computational Intelligence Systems* **2023**, *16*, 79. https://doi.org/10.1007/s44196-023-00250-5.
20. Lu, Y.; Cen, J.; Alkhoury Maroun, R.; Webb, B. Insect-inspired embodied visual route following. *Journal of Bionic Engineering* **2025**, *22*, 1167–1193. https://doi.org/10.1007/s42235-025-00695-8.
21. Li, F.; Guo, C.; Luo, B.; Zhang, H. Multi goals and multi scenes visual mapless navigation in indoor using meta-learning and scene priors. *Neurocomputing* **2021**, *449*, 368–377. https://doi.org/10.1016/j.neucom.2021.03.084.
22. Hu, Y.; Wang, S.; Xie, Y.; Zheng, S.; Shi, P.; Rudas, I.; Cheng, X. Deep reinforcement learning-based mapless navigation for mobile robot in unknown environment with local optima. *IEEE Robotics and Automation Letters* **2025**, *10*, 628–635. https://doi.org/10.1109/LRA.2024.3511437.
23. Zhang, S.; Tang, W.; Li, P.; Zha, F. Mapless path planning for mobile robot based on improved deep deterministic policy gradient algorithm. *Sensors* **2024**, *24*, 5667. https://doi.org/10.3390/s24175667.
24. Gao, J.; Pang, X.; Liu, Q.; Li, Y. Hierarchical reinforcement learning for safe mapless navigation with congestion estimation, 2025, [arXiv:cs.RO/2503.12036].
25. Zhao, X.; Cai, W.; Tang, L.; Wang, T. ImagineNav: Prompting vision-language models as embodied navigator through scene imagination, 2024, [arXiv:cs.RO/2410.09874].
26. Bektaş, K.; Bozma, H.I. APF-RL: Safe mapless navigation in unknown environments. In Proceedings of the 2022 International Conference on Robotics and Automation (ICRA), 2022, pp. 7299–7305. https://doi.org/10.1109/ICRA46639.2022.9811537.
27. Costa de Jesus, J.; Kich, V.A.; Kolling, A.H.; Grando, R.B.; da Silva Guerra, R.; Drews-Jr, P.L.J. Image-based mapless navigation of a hybrid aerial-underwater vehicle using prioritized deep reinforcement learning. *Journal of Intelligent & Robotic Systems* **2025**, *111*, 27. https://doi.org/10.1007/s10846-024-02206-z.
28. Krizhevsky, A.; Sutskever, I.; Hinton, G.E. ImageNet classification with deep convolutional neural networks. In Proceedings of the Advances in Neural Information Processing Systems, 2012, Vol. 25.
29. Brenner, M.; Reyes, N.H.; Susnjak, T.; Barczak, A.L.C. RGB-D and thermal sensor fusion: a systematic literature review. *IEEE Access* **2023**, *11*, 82410–82442. https://doi.org/10.1109/ACCESS.2023.3301119.
30. Elman, J.L. Finding structure in time. *Cognitive Science* **1990**, *14*, 179–211.
31. Mandic, D.P.; Chambers, J.A. *Recurrent Neural Networks for Prediction*; Wiley: Chichester, UK, 2001.
32. Contreras, L.; Negrete, M.; Yoshikai, T.; Hirose, Y.; Okada, H. A motion planning framework for multi-robot navigation in ROS 2. In Proceedings of the Proceedings of the 43rd Annual Conference of the Robotics Society of Japan, 2025.
33. Soviany, P.; Ionescu, R.T.; Rota, P.; Sebe, N. Curriculum learning: a survey. *arXiv preprint* **2021**, [2101.10382].